ARISS Clock - Helper

By: Ken McCaughey (N3FZX)
On: 2025-03-14
Ver 3.0.0

This file provides helpful information for users new to Python to help run the `ARISS Clock`. You do not need to know Python to run this!

The `README` file has information on how the tool works. This file has information on how to setup Python and run the tool. This is were known issues are documented and additional tips reside.

Contents

Python

Python is an interpreted programming language. This means it is not compiled into machine before being run. Python converts each line of code into machine code as it run the program. Python scripts, or programs, are initiated from the command line on your computer.

Python comes with Linux and Raspberry Pi's. Macs often have Python, but it is not configured for user projects. Windows doesn't come with Python.

For more information on Python, go to: https://www.python.org/

Linux and Raspberry Pi systems also usually come with Libre Office, which includes the fonts used by `ARISS Clock`. Thonny is in the Raspberry Pi software repository and most Linux repositories. These systems are generally easier to setup to use this tool.

The sections below assume Python is already installed on your system. If you are a beginner or Python is not installed, look here: https://wiki.python.org/moin/BeginnersGuide/Download

Or, skip the the next section and consider installing Thonny.

Python Test Script

A test script called `ARISS_Clock_Python_Test.py` is included. This simple test script will help you assess if the needed libraries needed are installed.

Using terminal window, change to the folder where this script resides. Use `cd` command to change directories. Use `ls` or `dir` to list files and verify this file is present.

At command line run this script:

```
python3 ARISS_Clock_Python_Test.py
```

If there are no issues the output will resemble this:

```
Python script: ARISS_Clock_Python_Test.py
    V.: 1.0.0
    By: Ken McCaughey, N3FZX
    On: 2025-03-14

Hello World!
The current date and time is 2025-03-15 15:46:43.670100

This is a simple test script for new Python users.
It is intended to make sure scripts can be executed.
This checks for Python libraries needed by ARISS Clock.

The libraries needed by ARISS Clock are present.

Success! Congratulations, you just ran a Python script.

Don't forget to install the two needed fonts.
```

The error message for missing libraries resembles the following:

```
Traceback (most recent call last):
File "<some_path>/ARISS_Clock_V3.0.0./ARISS_Clock.py", line xx, in <module>
import <library_name>
ModuleNotFoundError: No module named '<library_name>'
```

The script will stop after the first import error and tell which library is missing. If you are missing more than one, you will find out one-at-a-time. See below for more info on installing libraries.

Installing Python Libraries

This tool requires a number Python library that may not normally be included with a Python installation.

```
sys
os
platform
getopt
tkinter
time
datetime
```

Only install if Python complains of missing libraries. Then only install what's missing.

How libraries get added depended on your setup. In general they can be installed using the command line command: `pip install <name>`. If you are using a virtual environment (i.e. `venv`), execute the command in that folder with the virtual environment active. Another options you can try `python2 -m pip install <name>`.

Alternatively, use Thonny and its tool to install library packages. This is detailed below.

Running `ARISS_Clock.py`

To run the tool, open a terminal and at the command line enter:

```
python3 ARISS_Clock.py
```

Thonny Python IDE

Thonny is a decent basic Python Integrated Development Environment (IDE). It is free and runs on Linux (and Raspberry Pi's), Macs, and Windows. It also bring along its own Python installation. Installing Thonny gets you a good tool and Python in one step. This is recommended for Python beginners. The software may be in your machine's software repository (or store). Or the files for your OS can be found at https://thonny.org/. The wiki for Thonny can be found at https://github.com/thonny/thonny/wiki.

For Windows and the Mac it is best to install for current user only, not all users. This should not require admin privileges.

Once installed, turn on the file viewer. In Thonny, from the main toolbar click on `View` then click to add a check mark for `Files`. It will add a sub-window to the left side. When you are running the script this should be set to your working directory with all the ARISS Moderator Script files.

Installing Thonny on a Mac

A good guide for installing Thonny on Mac is at the link below. It also has some instructions for adding libraries.

https://www2.seas.gwu.edu/~cs4all/1012/editor-install/thonny-mac.html

Thonny Virtual Environment Setup

Starting with Python version 3.11, a virtual environment is required. This is in essence a local container of the Python files for users to use. It isolates any Python files the operating system may be using to protect your OS. Thonny supports the virtual environment. This needs to be setup only once for Thonny. It can use used for all your Python projects.

Note that the virtual environment setup is not required under Windows.

Start by making a Python project folder, i.e. `Python_Projects`. Within the folder create a new empty folder called `venv`. This will be the location for the user virtual environment, which will be setup below.

In Thonny, from the main toolbar click on `Tools` then `Options`. It will open a window. Select the `Interpreter` tab.

Which kind of interpreter... should be `Local Python 3`. If not click on the drop down menu and select `Local Python 3`.

At the bottom right of the window find `New virtual environment`. Click on that, and select the `venv` folder created above.

Now set Thonny to use that virtual environment. `Python executable` has a drop down menu. Click on it and find the path that corresponds to the `venv` folder. Note that the path will end with something like `.../Python_projects/venv/bin/python3`. Click on `OK` to close the `Thonny Options` window.

How To Add Python Libraries in Thonny

In Thonny, from the main toolbar click on `Tools` then `Manage packages...`. A window will open up. All the installed packages are listed in a column on the left. These are all packages only installed in the virtual environment.

In the box at the top you can enter the name of the package (or library) needed. It will provide a list of matches under `Search results`. Click on the one you need. It will then give you a more detailed description with an option to Install. Click on `Install`. A small window will appear as it is installed. Once complete it will appear on the list column on the left. There is also an option to `Uninstall`. Note that if it was already installed, there may be an options to `Upgrade` if a newer version has been released. Click `Close` when done.

Setup `ARISS_Clock.py` in Thonny

Download the Zipped package from GitHub at: https://github.com/twk6809/ARISS-Clock

Unzip the GitHub file `ARISS_Clock-main.zip`. Find the the `ARISS_Clock_V3.0.0` folder and copy to the `Python_projects` folder.

Running `ARISS_clock.py` in Thonny

It is possible to associate Python files (with the `.py` ext) with Thonny. The method varies with OS, so it is not included here. If you do this, a double click on any `.py` file can open it up in Thonny automatically.

Open Thonny and on the main toolbar click on `File`, then `Open` and work your way to the folder with the file `ARISS_Clock.py` and open the file. It will open the Python script in its own tab.

In the Thonny Files sub-window (left side) you should see all the ARISS Moderator Script files.

Any of the sub-windows in Thonny can be resized. Just grab the edges with the mouse and drag to suit.

Thonny can open and edit text files, such as the `ARISS_Clock_config.txt` file. It can open the `ARISS_Clock_readme.txt` file as well.

To run the script, just click on the `Run` button (green circle with right arrow) on the toolbar. Or on the toolbar click on the `Run` menu, then click on `Run current script...`.

To run with any of the command line switches (see the README) then it needs to be run from the shell. In the Thonny `Shell` sub-window at the prompt:

```
>>> %Run ARISS_Clock.py <options>

For example to have the Clocks above the timers enter:
>>> %Run ARISS_Clock.py -T

For the list of options enter:
>>> %Run ARISS_Clock.py -h
```

Then just minimize Thonny. If you close Thonny you close the ARISS_Clock too.

Creating a Native Executable

Native binaries are no longer included on the GitHub page for this project. You can create one with the steps below. This is not for the beginner. The information provided here is for those who which to experiment.

It is possible to make an executable for your operating system. You need to have Python installed. It can be done with Thonny. The advantage is that you can just run it without having to fuss this Python or Thonny. There is some limited portability to copy and run the executable on other machines. The disadvantage is that you need to create and test a new executable with every update that comes out.

Creating a native executable requires an extra file, `ARISS_logo_simple.ico`.

Install `pyinstaller`

To install the `pyinstaller` library needed by Thonny, from the main toolbar click on `Tools` then `Manage packages...`. In the search box, enter `pyinstaller`. Click on `pyinstaller` in the search results. Click on `Install`. Click `Close` when done.

Running `pyinstaller`

Below are the command line instructions for making a native binary executable for most operating systems. Below are the commands to be run at the command line. A bunch of messages will scroll by and, if successful, end with something similar to the following:

```
##### INFO: Building EXE from EXE-00.toc completed successfully.
```

This will also create a file called `ARISS_Clock.spec`. The file has the parameters from the last time the `pyinstaller` was run.

In all cases additional two folders will be created, `build` and `dist`. The files in the `build` do not need to be saved. The file in the `dist` folder is the native binary executable. This file should be copied to the with all the other `ARISS Clock` files. A shortcut can be made if desired, but setup to run in a terminal window.

To run the native executable, in a terminal window, enter `ARISS_Clock`.

Make a Linux (or Raspberry Pi) Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the `ARISS Clock` files.
- On Linux/Rasp Pi command line (all on one line):

```
pyinstaller --onefile -w -F -i "ARISS_logo_simple.ico" --add-data 'ARISS_logo.png:.' ARISS_Clock.py
```

Make a Windows Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the `ARISS Clock` files.
- On Windows command line (all on one line):

```
pyinstaller -w -F -i "ARISS_logo_simple.ico" --add-data ARISS_logo.png;. --add-data ARISS_logo_simple.ico;. ARISS_Clock.py
```

Make a Mac Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the `ARISS Clock` files.
- On Mac command line (all on one line):

```
pyinstaller -w -F -i "ARISS_logo_simple.ico" --add-data ARISS_logo.png;. --add-data ARISS_logo_simple.ico;. ARISS_Clock.py
```

Operating Suggestions

The following are some operating suggestions based on experience at K6DUE. We use this tool for situational awareness since we usually have multiple operators at the station for a scheduled ISS contact. Some of the information that `ARISS Clock` displays is also displayed by the satellite tracking software we use. However, it is in a small font and not so easy to read. We setup `ARISS Clock` to be nice and big on one of our computer monitors so everyone can see it clearly. We always check to make sure that the rise and set times we enter in the config file match what that satellite track software indicates.

Telebridge Stations

For use at telebridge stations I recommend the default settings. Usually moderator scripts have a timeline that is indexed to the local time at the event. Therefore it is important to set the Event Time Zone (ETZ) parameter. This helps to better follow along on the script and avoid real-time time math.

I would advise to always check that the Event Local Time displays matches the time at the event. This should be part of the moderator's pre-contact checklist.

The elapsed time counter can be useful to note/log the delay in making contact with ISS. Also can note significant drop outs or early end of a contact.

Moderators

For use at remote moderators I recommend the default settings. Usually moderator scripts have a timeline that is indexed to the local time at the event. Therefore it is important to set the Event Time Zone (ETZ) parameter. This helps to better follow along on the script and execute the program smoothly.

I would advise to always check that the Event Local Time displays matches the time at the event. This should be part of the pre-contact checklist.

Note that the clock window could also be included in a live stream if using conference software. But be cautious of network delays that could be large enough to create confusion. Time deltas of a second or two should not be an issue. If the delta is beyond that, I would discourage streaming the clock.

Event Locations and Schools

This could be good to have on display at the event. The people coordinating at the event should work with the mentor, moderator and/or telebridge station to be certain the correct times are in the configuration file.

It would be advisable to setup the `ARISS Clock` with a shortcut to the config file and the Clock with the desired options in advance. This makes it much easier to run during the event. Practicing using the tool is not to be overlooked.

This could be run on a Rasp Pi, as long as the Pi has its clock set correctly. Many models of the Rasp Pi require a Wifi connection to get network time. This may require some advanced planning. I would highly recommend it be tested well in advance.

A Rasp Pi can be a good choice for schools. The Rasp Pi OS has Python, and if it was setup with Libre Office, also has the fonts. These considerations can make it easier to employ.

For use locally at the event I would recommend running with the `-e` option (`python3 ARISS_Clock.py -e`). Local time and event local time would be the same and redundant. Alternatively, use the mouse and grab the bottom edge of the clock window and drag up until the Event Local Time Clock is not visible.

If display space is more limited, I would recommend reducing the clock window to only show the top two or three timers. First use the mouse grab the bottom of the clock window and adjust to desired window hight. Then grab one of the clock window sides and drag right/left to get desired width. Use these two means of adjustment to get the desired timers displayed.

To display additional information, click on the gold `ARISS Contact Clock` button at the top of the clock. This will open a window displaying the predicted rise and set times with a nice ARISS logo. Closing that window will not affect the clock.

Known Issues and Tips

If you get errors or have issues take a look here first.

If you update the config file, you must restart `ARISS Clock` for the new data to take affect.

If the fonts that this was designed around are not found, a window will pop up saying the fonts were not found. It will still run using substituted fonts, but the look may be sloppy and text might not fit properly. If this happens, install the fonts!

There is some error checking for the configuration file. The messages may not seem all that useful. Here are some things to look for:

- Do not use config files from older versions of `ARISS Clock`. They are not compatible. The config file specifies what version it goes with.
- A blank line that has spaces and no other text.
- Comments that don't start with a `#` character.
- Wrong date or time format.
- Spaces after the comma that separates the variable label and the value. (i.e. `RT, 2025-03-12 16:10:00`)
- Check for missing leading zeros in the date/time.
- The Rise date/time is BEFORE the Set date/time.
- There is more than one set of ETZ, RS, and ST data. Comment out unneeded ones.

There are some errors to the configuration file that can be missed. In that case the script uses default values. If the rise/set times seem very wrong, check the config file.

One sure way to recover from a corrupted config file is to delete it and rerun the `ARISS Clock`. It will inform you that no config file was found and a new fresh one will be created with default values. Edit the new file.

It is possible to run multiple instances of the clock. They can be run using different options. They can all use the same config file.

When running this from a terminal window (or command window, or DOS window), closing that window exits the program. Just minimize the terminal window.

For Linux and Rasp Pi users, you can run this and have it close the terminal window by adding `& exit` to the command (i.e. `python3 ARISS_Clock.py & exit`). This might work on MacOS too.

It is possible to create a shortcut to run this with whatever options you want. This is recommended for regular users.

If running in Thonny, make sure the `ARISS_Clock` tab is active before clicking on `Run`. If another tab, such as the config file is open and the active tab, it will try to run what is in the form, which is not Python, and generate lots of errors. This is easy to do! Just make the tab with the script the active one an run it.

Be aware that if you create a native executable for Windows and/or the Mac it might trip virus protection software and quarantine it.